

```
``html
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <!-- Отключаем зум и скролл для мобилок, чтобы играть было комфортно -->
  <meta name="viewport" content="width=device-width, initial-scale=1.0,
maximum-scale=1.0, user-scalable=no">
  <title>Abyssal Purge</title>
  <style>
    /* Мрачный стиль: никаких лишних цветов, всё черное и жесткое */
    body {
      margin: 0;
      padding: 0;
      background-color: #050505;
      color: #fff;
      font-family: 'Courier New', Courier, monospace;
      overflow: hidden;
      touch-action: none; /* Запрещаем жесты браузера */
      user-select: none;
      -webkit-user-select: none;
    }

    #gameCanvas {
      display: block;
      width: 100vw;
      height: 100vh;
    }

    /* UI Элементы */
    #uiLayer {
      position: absolute;
      top: 0;
      left: 0;
      width: 100%;
      height: 100%;
      pointer-events: none; /* Клики проходят сквозь UI к канвасу */
    }

    .hud-bar {
      position: absolute;
      left: 50%;
      transform: translateX(-50%);
      width: 80%;
      max-width: 400px;
      background: #222;
      border: 2px solid #555;
      border-radius: 4px;
```

```

    overflow: hidden;
}

#hpBarContainer { top: 20px; height: 20px; border-color: #511;}
#hpBar { width: 100%; height: 100%; background: #c00; transition: width 0.1s; }

#xpBarContainer { top: 45px; height: 10px; border-color: #151;}
#xpBar { width: 0%; height: 100%; background: #0c0; transition: width 0.1s; }

#scoreDisplay {
    position: absolute;
    top: 60px;
    left: 50%;
    transform: translateX(-50%);
    font-size: 20px;
    font-weight: bold;
    color: #ccc;
    text-shadow: 2px 2px 0 #000;
}

#waveDisplay {
    position: absolute;
    top: 20px;
    right: 20px;
    font-size: 18px;
    color: #aaa;
}

/* Меню смерти и старта */
.overlay-menu {
    position: absolute;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    background: rgba(0, 0, 0, 0.85);
    display: flex;
    flex-direction: column;
    justify-content: center;
    align-items: center;
    pointer-events: auto;
    z-index: 100;
}

h1 {
    color: #d00;
    font-size: 40px;
    text-transform: uppercase;

```

```
    letter-spacing: 5px;
    margin-bottom: 10px;
    text-shadow: 0 0 10px #f00;
    text-align: center;
}
```

```
p { color: #888; margin-bottom: 30px; text-align: center; max-width: 80%;}
```

```
button {
    background: #222;
    color: #fff;
    border: 2px solid #555;
    padding: 15px 30px;
    font-size: 20px;
    font-family: inherit;
    cursor: pointer;
    border-radius: 5px;
    transition: 0.2s;
    text-transform: uppercase;
}
```

```
button:active { background: #555; transform: scale(0.95); }
```

```
/* Экран повышения уровня (Прокачка/Приколы) */
#levelUpMenu {
    display: none;
    background: rgba(10, 10, 10, 0.95);
}
```

```
.upgrade-cards {
    display: flex;
    flex-direction: column;
    gap: 15px;
    width: 90%;
    max-width: 400px;
}
```

```
.card {
    background: #1a1a1a;
    border: 2px solid #444;
    padding: 15px;
    border-radius: 8px;
    cursor: pointer;
    text-align: left;
}
```

```
.card:active { background: #333; border-color: #777;}
.card-title { color: #d0d; font-size: 18px; font-weight: bold; margin-bottom: 5px; }
.card-desc { color: #aaa; font-size: 14px; }
```

```

/* Скрываем элементы */
.hidden { display: none !important; }

</style>
</head>
<body>

<!-- Игровой холст -->
<canvas id="gameCanvas"></canvas>

<!-- Интерфейс поверх холста -->
<div id="uiLayer">
  <div id="hpBarContainer" class="hud-bar"><div id="hpBar"></div></div>
  <div id="xpBarContainer" class="hud-bar"><div id="xpBar"></div></div>
  <div id="scoreDisplay">ЧЕРЕПОВ: 0</div>
  <div id="waveDisplay">УРОВЕНЬ: 1</div>
</div>

<!-- Главное меню -->
<div id="startMenu" class="overlay-menu">
  <h1>ABYSSAL PURGE</h1>
  <p>Мрачный мир. Только ты, пушка и толпы тварей.<br><br><b>Управление:</b>
  Левая часть экрана — бег. Правая часть экрана — стрельба. Двойной тап слева —
  РЫБОК.</p>
  <button onclick="startGame()">Устроить резню</button>
</div>

<!-- Меню смерти -->
<div id="deathMenu" class="overlay-menu hidden">
  <h1>ПОТРАЧЕНО</h1>
  <p id="deathStats">Ты убил: 0 тварей</p>
  <button onclick="resetGame()">Восстать из мертвых</button>
</div>

<!-- Меню Level Up -->
<div id="levelUpMenu" class="overlay-menu">
  <h1 style="color: #0d0; text-shadow: 0 0 10px #0f0;">НОВЫЙ УРОВЕНЬ</h1>
  <p>Выбери прикол для выживания:</p>
  <div class="upgrade-cards" id="upgradeCards">
    <!-- Карточки генерируются через JS -->
  </div>
</div>

<script>
/**
 * НАСТРОЙКИ И ЯДРО ИГРЫ
 */

```

```
const canvas = document.getElementById('gameCanvas');
const ctx = canvas.getContext('2d', { alpha: false }); // alpha: false для оптимизации
```

```
let width, height;
function resize() {
    width = window.innerWidth;
    height = window.innerHeight;
    canvas.width = width;
    canvas.height = height;
}
window.addEventListener('resize', resize);
resize();
```

```
// Состояния игры
let gameState = 'START'; // START, PLAYING, LEVELUP, DEAD
let lastTime = 0;
```

```
// Глобальные массивы сущностей
let player;
let bullets = [];
let enemies = [];
let particles = [];
let bloodDecals = [];
let items = [];
let floatingTexts = [];
```

```
// Камера (следит за игроком)
let camera = { x: 0, y: 0 };
let screenShake = 0;
```

```
// Переменные прогресса
let score = 0;
let level = 1;
let currentXP = 0;
let xpToNextLevel = 10;
let timeSurvived = 0;
let spawnRate = 2000; // мс между спауном
let lastSpawn = 0;
```

```
/**
 * УПРАВЛЕНИЕ (Два виртуальных стика + рывок)
 */
const input = {
    leftJoy: { active: false, id: null, originX: 0, originY: 0, dirX: 0, dirY: 0, strength: 0 },
    rightJoy: { active: false, id: null, originX: 0, originY: 0, dirX: 0, dirY: 0, strength: 0 },
    dashReady: true,
    lastLeftTap: 0
};
```

```

// Обработка касаний
canvas.addEventListener('touchstart', handleTouch, {passive: false});
canvas.addEventListener('touchmove', handleTouch, {passive: false});
canvas.addEventListener('touchend', handleTouchEnd);
canvas.addEventListener('touchcancel', handleTouchEnd);

function handleTouch(e) {
    if (gameState !== 'PLAYING') return;
    e.preventDefault();

    for (let i = 0; i < e.touches.length; i++) {
        let t = e.touches[i];
        let half = width / 2;

        // Левая половина - Движение
        if (t.clientX < half) {
            if (!input.leftJoy.active) {
                // Проверка на двойной тап для рывка (Dash)
                let now = Date.now();
                if (now - input.lastLeftTap < 300 && input.dashReady && player) {
                    player.dash();
                }
                input.lastLeftTap = now;

                input.leftJoy.active = true;
                input.leftJoy.id = t.identifier;
                input.leftJoy.originX = t.clientX;
                input.leftJoy.originY = t.clientY;
            }
            if (input.leftJoy.id === t.identifier) {
                calcJoystick(input.leftJoy, t.clientX, t.clientY, 50);
            }
        }
        // Правая половина - Стрельба
        else {
            if (!input.rightJoy.active) {
                input.rightJoy.active = true;
                input.rightJoy.id = t.identifier;
                input.rightJoy.originX = t.clientX;
                input.rightJoy.originY = t.clientY;
            }
            if (input.rightJoy.id === t.identifier) {
                calcJoystick(input.rightJoy, t.clientX, t.clientY, 60);
            }
        }
    }
}

```

```

function handleTouchEnd(e) {
    e.preventDefault();
    // Проверяем какие пальцы убрали
    let leftStillActive = false;
    let rightStillActive = false;

    for (let i = 0; i < e.touches.length; i++) {
        if (e.touches[i].identifier === input.leftJoy.id) leftStillActive = true;
        if (e.touches[i].identifier === input.rightJoy.id) rightStillActive = true;
    }

    if (!leftStillActive) {
        input.leftJoy.active = false;
        input.leftJoy.dirX = 0;
        input.leftJoy.dirY = 0;
    }
    if (!rightStillActive) {
        input.rightJoy.active = false;
        input.rightJoy.dirX = 0;
        input.rightJoy.dirY = 0;
    }
}

function calcJoystick(joy, currentX, currentY, maxDist) {
    let dx = currentX - joy.originX;
    let dy = currentY - joy.originY;
    let dist = Math.hypot(dx, dy);

    // Нормализация
    if (dist > 0) {
        joy.dirX = dx / dist;
        joy.dirY = dy / dist;
    } else {
        joy.dirX = 0;
        joy.dirY = 0;
    }

    // Ограничение силы
    joy.strength = Math.min(dist / maxDist, 1);
}

/**
 * БАЗОВЫЕ КЛАССЫ ИГРЫ
 */

// Пул перков/приколов
const PERKS = [

```

```

    { id: 'hp', name: 'Свинец в крови', desc: '+50 Макс. Здоровья, лечит на 50.', type: 'stats'
  },
  { id: 'speed', name: 'Кроссовки с рынка', desc: '+15% к скорости бега.', type: 'stats' },
  { id: 'dmg', name: 'Озлобленность', desc: '+25% к урону пушки.', type: 'stats' },
  { id: 'rate', name: 'Кофеиновый передоз', desc: '+20% к скорости стрельбы.', type: 'stats'
  },
  { id: 'pierce', name: 'Пробитие насквозь', desc: 'Пули прошивают на 1 врага больше.',
type: 'mechanic' },
  { id: 'vampire', name: 'Вампиризм', desc: '5% шанс восстановить 5 HP при убийстве.',
type: 'mechanic' },
  { id: 'multishot', name: 'Косоглазие', desc: 'Выстреливает 1 доп. пулю веером.', type:
'mechanic' },
  { id: 'saw', name: 'Пила "Дружба"', desc: 'Иногда вылетает циркулярная пила,
кромсающая всех.', type: 'weapon' },
  { id: 'explosive', name: 'Китайская петарда', desc: 'Пули имеют шанс 15% взорваться
при попадании.', type: 'mechanic' }
];

```

```

class Player {
  constructor() {
    this.x = 0;
    this.y = 0;
    this.vx = 0;
    this.vy = 0;
    this.radius = 16;

    // Статы
    this.maxHealth = 100;
    this.health = 100;
    this.speed = 4.5;
    this.damageMult = 1;
    this.fireRateMult = 1;
    this.pierceCount = 0;
    this.multishot = 1;
    this.vampire = false;
    this.explosive = false;
    this.hasSaw = false;

    // Стрельба
    this.lastShot = 0;
    this.baseFireDelay = 200; // мс

    // Рывок (Dash)
    this.isDashing = false;
    this.dashTimer = 0;
    this.dashCooldown = 0;
    this.dashDirX = 0;
    this.dashDirY = 0;
  }
}

```



```

// Визуал
this.color = '#ddd';
this.aimAngle = 0;
this.invulnerableTimer = 0;
}

update(dt) {
    // Уменьшение таймеров
    if (this.invulnerableTimer > 0) this.invulnerableTimer -= dt;
    if (this.dashCooldown > 0) this.dashCooldown -= dt;

    // Обработка рывка
    if (this.isDashing) {
        this.dashTimer -= dt;
        this.vx = this.dashDirX * this.speed * 4;
        this.vy = this.dashDirY * this.speed * 4;
        // Частицы тени
        addParticle(this.x, this.y, 0, 0, '#666', 150, 10);

        if (this.dashTimer <= 0) {
            this.isDashing = false;
        }
    } else {
        // Обычное движение от стика
        if (input.leftJoy.active) {
            // Плавный разгон
            this.vx += (input.leftJoy.dirX * this.speed - this.vx) * 0.2;
            this.vy += (input.leftJoy.dirY * this.speed - this.vy) * 0.2;
        } else {
            // Трение
            this.vx *= 0.8;
            this.vy *= 0.8;
        }
    }
}

// Применяем скорость
this.x += this.vx * (dt / 16);
this.y += this.vy * (dt / 16);

// Стрельба
if (input.rightJoy.active && !this.isDashing) {
    this.aimAngle = Math.atan2(input.rightJoy.dirY, input.rightJoy.dirX);
    let now = Date.now();
    let delay = this.baseFireDelay / this.fireRateMult;

    if (now - this.lastShot > delay) {
        this.shoot();
    }
}

```

```

        this.lastShot = now;
    }
}

updateHUD();
}

shoot() {
    // Отдача
    this.vx -= Math.cos(this.aimAngle) * 2;
    this.vy -= Math.sin(this.aimAngle) * 2;
    addScreenShake(3);

    let damage = 25 * this.damageMult;

    // Мультивыстрел (веер)
    let spread = 0.2; // угол разброса
    let startAngle = this.aimAngle - (spread * (this.multishot - 1)) / 2;

    for(let i=0; i<this.multishot; i++) {
        let angle = startAngle + i * spread;
        // Добавляем небольшой случайный разброс
        angle += (Math.random() - 0.5) * 0.1;

        bullets.push(new Bullet(this.x, this.y, angle, 15, damage, this.pierceCount, '#fd0'));
    }

    // Прикол: Пила Дружба
    if (this.hasSaw && Math.random() < 0.15) {
        bullets.push(new SawBlade(this.x, this.y, this.aimAngle, 8, damage * 2));
    }

    // Гильзы
    let casingAngle = this.aimAngle + Math.PI/2 + (Math.random()-0.5);
    addParticle(this.x, this.y, Math.cos(casingAngle)*3, Math.sin(casingAngle)*3, '#a82',
500, 3, true);
}

dash() {
    if (this.dashCooldown > 0) return;
    this.isDashing = true;
    this.dashTimer = 200; // мс
    this.dashCooldown = 1500;

    // Направление рывка: если стик нажат, то туда. Иначе туда, куда смотрим.
    if (input.leftJoy.active) {
        this.dashDirX = input.leftJoy.dirX;
        this.dashDirY = input.leftJoy.dirY;
    }
}

```

```

    } else {
        this.dashDirX = Math.cos(this.aimAngle);
        this.dashDirY = Math.sin(this.aimAngle);
    }
}

takeDamage(amt) {
    if (this.invulnerableTimer > 0 || this.isDashing) return;

    this.health -= amt;
    this.invulnerableTimer = 500; // Полкадры неуязвимости
    addScreenShake(10);

    // Кровища игрока
    for(let i=0; i<15; i++) {
        let angle = Math.random() * Math.PI * 2;
        let speed = Math.random() * 5 + 2;
        addParticle(this.x, this.y, Math.cos(angle)*speed, Math.sin(angle)*speed, '#d00', 800,
4, true);
    }

    if (this.health <= 0) {
        this.health = 0;
        die();
    }
}

heal(amt) {
    this.health = Math.min(this.maxHealth, this.health + amt);
    addFloatingText(this.x, this.y - 20, `+${amt}`, '#0f0');
}

draw(ctx) {
    // Мигание при уроне
    if (this.invulnerableTimer > 0 && Math.floor(this.invulnerableTimer / 50) % 2 === 0)
return;

    ctx.save();
    ctx.translate(this.x, this.y);
    ctx.rotate(this.aimAngle);

    // Пушка
    ctx.fillStyle = '#999';
    ctx.fillRect(this.radius - 5, -4, 18, 8);

    // Тепло (мрачное, в стиле Hatred - серое с черным контуром)
    ctx.beginPath();
    ctx.arc(0, 0, this.radius, 0, Math.PI * 2);

```

```

        ctx.fillStyle = this.color;
        ctx.fill();
        ctx.lineWidth = 3;
        ctx.strokeStyle = '#222';
        ctx.stroke();

        ctx.restore();
    }
}

class Bullet {
    constructor(x, y, angle, speed, damage, pierce, color) {
        this.x = x;
        this.y = y;
        this.vx = Math.cos(angle) * speed;
        this.vy = Math.sin(angle) * speed;
        this.damage = damage;
        this.pierce = pierce; // Сколько врагов прошивает
        this.color = color;
        this.radius = 3;
        this.life = 2000; // Мертва через 2 сек
        this.hitIds = new Set(); // Кого уже ударила, чтобы не бить дважды за фрейм
    }

    update(dt) {
        this.x += this.vx * (dt / 16);
        this.y += this.vy * (dt / 16);
        this.life -= dt;

        // Трейл
        if (Math.random() > 0.5) {
            addParticle(this.x, this.y, 0, 0, '#fa0', 100, 2);
        }
    }

    draw(ctx) {
        ctx.beginPath();
        ctx.arc(this.x, this.y, this.radius, 0, Math.PI * 2);
        ctx.fillStyle = this.color;
        ctx.fill();
    }
}

// Уникальная пушка: Пила
class SawBlade extends Bullet {
    constructor(x, y, angle, speed, damage) {
        super(x, y, angle, speed, damage, 999, '#aaa');
        this.radius = 12;
    }
}

```

```

    this.life = 3000;
    this.rotation = 0;
}
update(dt) {
    super.update(dt);
    this.rotation += 0.5;
    // Самонаведение небольшое на ближайшего
    let target = getClosestEnemy(this.x, this.y, 200);
    if (target) {
        let dx = target.x - this.x;
        let dy = target.y - this.y;
        let len = Math.hypot(dx, dy);
        this.vx += (dx/len) * 0.5;
        this.vy += (dy/len) * 0.5;
        // Ограничение скорости
        let curSpeed = Math.hypot(this.vx, this.vy);
        if (curSpeed > 8) {
            this.vx = (this.vx / curSpeed) * 8;
            this.vy = (this.vy / curSpeed) * 8;
        }
    }
}
draw(ctx) {
    ctx.save();
    ctx.translate(this.x, this.y);
    ctx.rotate(this.rotation);
    ctx.beginPath();
    // Рисуем звезду (пилу)
    for(let i=0; i<8; i++) {
        let r = i%2==0 ? this.radius : this.radius/2;
        let a = (i/8) * Math.PI*2;
        ctx.lineTo(Math.cos(a)*r, Math.sin(a)*r);
    }
    ctx.closePath();
    ctx.fillStyle = '#bbb';
    ctx.fill();
    ctx.strokeStyle = '#f00';
    ctx.stroke();
    ctx.restore();
}
}

```

```

class Enemy {
    constructor(x, y, type) {
        this.x = x;
        this.y = y;
        this.type = type; // 'walker', 'runner', 'tank'
    }
}

```

```

this.vx = 0;
this.vy = 0;
this.knockbackX = 0;
this.knockbackY = 0;

if (type === 'runner') {
    this.health = 30;
    this.maxHealth = 30;
    this.speed = 3.5;
    this.radius = 12;
    this.color = '#555';
    this.damage = 10;
} else if (type === 'tank') {
    this.health = 200;
    this.maxHealth = 200;
    this.speed = 1.2;
    this.radius = 24;
    this.color = '#333';
    this.damage = 30;
} else {
    // walker
    this.health = 60;
    this.maxHealth = 60;
    this.speed = 2.2;
    this.radius = 16;
    this.color = '#444';
    this.damage = 15;
}

// Скалирование от уровня
this.health += level * 10;
this.maxHealth = this.health;
}

update(dt) {
    // Логика следования за игроком
    let dx = player.x - this.x;
    let dy = player.y - this.y;
    let dist = Math.hypot(dx, dy);

    if (dist > 0) {
        this.vx = (dx / dist) * this.speed;
        this.vy = (dy / dist) * this.speed;
    }

    // Применяем отбрасывание
    this.x += (this.vx + this.knockbackX) * (dt / 16);
    this.y += (this.vy + this.knockbackY) * (dt / 16);
}

```

```

// Трение для отбрасывания
this.knockbackX *= 0.85;
this.knockbackY *= 0.85;

// Нанесение урона игроку
if (dist < this.radius + player.radius) {
    player.takeDamage(this.damage);
    // Немного отталкиваемся от игрока
    this.knockbackX = -(dx / dist) * 5;
    this.knockbackY = -(dy / dist) * 5;
}
}

takeDamage(amt, knockDirX, knockDirY) {
    this.health -= amt;
    // Отбрасывание
    this.knockbackX = knockDirX * 8;
    this.knockbackY = knockDirY * 8;

    // Кровь (брызги)
    for(let i=0; i<3; i++) {
        let spreadAngle = Math.atan2(knockDirY, knockDirX) + (Math.random()-0.5);
        let speed = Math.random() * 4 + 2;
        addParticle(this.x, this.y, Math.cos(spreadAngle)*speed,
Math.sin(spreadAngle)*speed, '#900', 600, 3, true);
    }

    // Текст урона
    addFloatingText(this.x, this.y, Math.floor(amt), '#fff');

    if (this.health <= 0) {
        this.die();
        return true;
    }
    return false;
}

die() {
    score++;
    gainXP(1);

    // Вампиризм
    if (player.vampire && Math.random() < 0.05) {
        player.heal(5);
    }

    // Жестокая лужа крови (Декаль)

```

```

addBloodDecal(this.x, this.y, this.radius * 1.5);

// Взрыв ошметков
for(let i=0; i<10; i++) {
    let angle = Math.random() * Math.PI * 2;
    let speed = Math.random() * 6;
    addParticle(this.x, this.y, Math.cos(angle)*speed, Math.sin(angle)*speed, '#700',
1000, 5, true);
}

// Шанс дропа аптечки (прикол: "Подорожник")
if (Math.random() < 0.03) {
    items.push(new Item(this.x, this.y, 'medkit'));
}
}

draw(ctx) {
    ctx.beginPath();
    ctx.arc(this.x, this.y, this.radius, 0, Math.PI * 2);
    ctx.fillStyle = this.color;
    ctx.fill();
    ctx.lineWidth = 2;
    ctx.strokeStyle = '#111';
    ctx.stroke();

    // Злые глаза (в сторону игрока)
    let angle = Math.atan2(player.y - this.y, player.x - this.x);
    ctx.save();
    ctx.translate(this.x, this.y);
    ctx.rotate(angle);
    ctx.fillStyle = '#f00';
    ctx.shadowBlur = 10;
    ctx.shadowColor = '#f00';
    ctx.beginPath();
    ctx.arc(this.radius/2, -4, 3, 0, Math.PI*2);
    ctx.arc(this.radius/2, 4, 3, 0, Math.PI*2);
    ctx.fill();
    ctx.restore();
}
}

class Item {
    constructor(x, y, type) {
        this.x = x;
        this.y = y;
        this.type = type; // Пока только 'medkit'
        this.radius = 10;
        this.bobTime = Math.random() * 10;
    }
}

```



```

}
update(dt) {
    this.bobTime += dt * 0.005;
    let dx = player.x - this.x;
    let dy = player.y - this.y;
    if (Math.hypot(dx, dy) < this.radius + player.radius) {
        this.collect();
        return true; // remove
    }
    return false;
}
collect() {
    if (this.type === 'medkit') {
        player.heal(30);
        addFloatingText(this.x, this.y, "Подорожник!", '#0f0');
    }
}
draw(ctx) {
    let offsetY = Math.sin(this.bobTime) * 3;
    ctx.fillStyle = '#fff';
    ctx.fillRect(this.x - 8, this.y - 8 + offsetY, 16, 16);
    ctx.fillStyle = '#f00';
    // Крест
    ctx.fillRect(this.x - 2, this.y - 6 + offsetY, 4, 12);
    ctx.fillRect(this.x - 6, this.y - 2 + offsetY, 12, 4);
}
}

```

```

class Particle {
    constructor(x, y, vx, vy, color, life, size, friction=false) {
        this.x = x;
        this.y = y;
        this.vx = vx;
        this.vy = vy;
        this.color = color;
        this.maxLife = life;
        this.life = life;
        this.size = size;
        this.friction = friction;
    }
    update(dt) {
        this.x += this.vx * (dt / 16);
        this.y += this.vy * (dt / 16);
        if (this.friction) {
            this.vx *= 0.9;
            this.vy *= 0.9;
        }
        this.life -= dt;
    }
}

```

```

    }
    draw(ctx) {
        let alpha = Math.max(0, this.life / this.maxLife);
        ctx.globalAlpha = alpha;
        ctx.fillStyle = this.color;
        ctx.beginPath();
        ctx.arc(this.x, this.y, this.size, 0, Math.PI*2);
        ctx.fill();
        ctx.globalAlpha = 1;
    }
}

function addParticle(x, y, vx, vy, color, life, size, friction) {
    particles.push(new Particle(x, y, vx, vy, color, life, size, friction));
}

function addBloodDecal(x, y, size) {
    bloodDecals.push({
        x: x + (Math.random()-0.5)*10,
        y: y + (Math.random()-0.5)*10,
        r: size + Math.random()*5,
        alpha: 0.6 + Math.random()*0.4
    });
    // Лимит декалей чтобы не лагало
    if (bloodDecals.length > 300) bloodDecals.shift();
}

function addFloatingText(x, y, text, color) {
    floatingTexts.push({ x, y, text, color, life: 1000, maxLife: 1000, vy: -1 });
}

function addScreenShake(intensity) {
    screenShake = Math.max(screenShake, intensity);
}

function getClosestEnemy(x, y, maxRange) {
    let closest = null;
    let minDist = maxRange;
    for(let e of enemies) {
        let d = Math.hypot(e.x - x, e.y - y);
        if (d < minDist) {
            minDist = d;
            closest = e;
        }
    }
    return closest;
}

```

```
/**
```

```
 * ИГРОВАЯ ЛОГИКА
```

```
 */
```

```
function spawnEnemies() {  
  let now = Date.now();  
  if (now - lastSpawn > Math.max(500, spawnRate - level * 100)) {  
    lastSpawn = now;  
  
    // Спавним за экраном  
    let angle = Math.random() * Math.PI * 2;  
    let dist = Math.max(width, height) / 2 + 50;  
    let sx = player.x + Math.cos(angle) * dist;  
    let sy = player.y + Math.sin(angle) * dist;  
  
    let type = 'walker';  
    let r = Math.random();  
    if (level > 2 && r < 0.2) type = 'runner';  
    else if (level > 4 && r > 0.9) type = 'tank';  
  
    enemies.push(new Enemy(sx, sy, type));  
  }  
}
```

```
function explosion(x, y, radius, damage) {  
  addScreenShake(15);  
  // Визуал взрыва  
  addParticle(x, y, 0, 0, '#f80', 200, radius, false);  
  for(let i=0; i<20; i++) {  
    let a = Math.random()*Math.PI*2;  
    let s = Math.random()*10;  
    addParticle(x, y, Math.cos(a)*s, Math.sin(a)*s, '#fa0', 500, 4, true);  
  }  
  
  // Урон по области  
  for(let i = enemies.length-1; i>=0; i--) {  
    let e = enemies[i];  
    let d = Math.hypot(e.x - x, e.y - y);  
    if (d < radius) {  
      let dx = (e.x - x)/d;  
      let dy = (e.y - y)/d;  
      if (e.takeDamage(damage, dx, dy)) {  
        enemies.splice(i, 1);  
      }  
    }  
  }  
}
```

```

function gainXP(amt) {
  currentXP += amt;
  if (currentXP >= xpToNextLevel) {
    currentXP -= xpToNextLevel;
    levelUp();
  }
}

function levelUp() {
  gameState = 'LEVELUP';
  level++;
  xpToNextLevel = Math.floor(xpToNextLevel * 1.5);
  player.heal(20); // Немного хилим при левел апе

  // Выбираем 3 случайных перка
  let shuffled = [...PERKS].sort(() => 0.5 - Math.random());
  let choices = shuffled.slice(0, 3);

  let container = document.getElementById('upgradeCards');
  container.innerHTML = "";

  choices.forEach(perk => {
    let div = document.createElement('div');
    div.className = 'card';
    div.innerHTML = `<div class="card-title">${perk.name}</div><div
class="card-desc">${perk.desc}</div>`;
    div.onclick = () => applyPerk(perk);
    container.appendChild(div);
  });

  document.getElementById('levelUpMenu').classList.remove('hidden');
  document.getElementById('levelUpMenu').style.display = 'flex';
}

function applyPerk(perk) {
  switch(perk.id) {
    case 'hp': player.maxHealth += 50; player.heal(50); break;
    case 'speed': player.speed *= 1.15; break;
    case 'dmg': player.damageMult *= 1.25; break;
    case 'rate': player.fireRateMult *= 1.20; break;
    case 'pierce': player.pierceCount += 1; break;
    case 'vampire': player.vampire = true; break;
    case 'multishot': player.multishot += 1; break;
    case 'saw': player.hasSaw = true; break;
    case 'explosive': player.explosive = true; break;
  }

  document.getElementById('levelUpMenu').style.display = 'none';
}

```

```

    input.leftJoy.active = false; // сбрасываем стики чтоб не залипли
    input.rightJoy.active = false;
    gameState = 'PLAYING';
    lastTime = performance.now(); // чтобы не было скачка дельты
}

function updateHUD() {
    document.getElementById('hpBar').style.width = Math.max(0, (player.health /
player.maxHealth) * 100) + '%';
    document.getElementById('xpBar').style.width = Math.min(100, (currentXP /
xpToNextLevel) * 100) + '%';
    document.getElementById('scoreDisplay').innerText = `ЧЕРЕПОВ: ${score}`;
    document.getElementById('waveDisplay').innerText = `УРОВЕНЬ: ${level}`;
}

function die() {
    gameState = 'DEAD';
    document.getElementById('deathStats').innerText = `Ты пережил волн: ${level}\nТы
искрошил тварей: ${score}\n\nЗло всегда побеждает.`;
    document.getElementById('deathMenu').classList.remove('hidden');
}

/**
 * ГЛАВНЫЙ ИГРОВОЙ ЦИКЛ (ОТРИСОВКА И АПДЕЙТ)
 */
function gameLoop(timestamp) {
    if (gameState === 'PLAYING') {
        let dt = timestamp - lastTime;
        if (dt > 100) dt = 16; // Защита от долгой паузы
        lastTime = timestamp;

        update(dt);
        draw();
    }
    requestAnimationFrame(gameLoop);
}

function update(dt) {
    player.update(dt);
    spawnEnemies();

    // Обновление пуль
    for (let i = bullets.length - 1; i >= 0; i--) {
        let b = bullets[i];
        b.update(dt);
        if (b.life <= 0) {
            bullets.splice(i, 1);
            continue;
        }
    }
}

```

```

}

// Коллизия пули с врагами
let hitSomething = false;
for (let j = enemies.length - 1; j >= 0; j--) {
  let e = enemies[j];
  if (b.hitIds.has(e)) continue; // Уже прошили его

  if (Math.hypot(b.x - e.x, b.y - e.y) < b.radius + e.radius) {
    b.hitIds.add(e);

    // Взрывные пули (Прикол)
    if (player.explosive && Math.random() < 0.15) {
      explosion(b.x, b.y, 60, b.damage * 1.5);
      hitSomething = true;
    } else {
      // Обычный урон
      let dx = b.vx / Math.hypot(b.vx, b.vy);
      let dy = b.vy / Math.hypot(b.vx, b.vy);
      if (e.takeDamage(b.damage, dx, dy)) {
        enemies.splice(j, 1);
      }
    }
  }

  if (b.hitIds.size > b.pierce) {
    hitSomething = true;
    break;
  }
}
}
if (hitSomething) bullets.splice(i, 1);
}

// Враги
for (let i = enemies.length - 1; i >= 0; i--) {
  enemies[i].update(dt);

  // Коллизия врагов между собой (простое расталкивание)
  for (let j = i - 1; j >= 0; j--) {
    let e1 = enemies[i];
    let e2 = enemies[j];
    let dx = e2.x - e1.x;
    let dy = e2.y - e1.y;
    let dist = Math.hypot(dx, dy);
    let minDist = e1.radius + e2.radius;
    if (dist < minDist && dist > 0) {
      let overlap = minDist - dist;
      let nx = dx / dist;

```

```

        let ny = dy / dist;
        e1.x -= nx * overlap * 0.5;
        e1.y -= ny * overlap * 0.5;
        e2.x += nx * overlap * 0.5;
        e2.y += ny * overlap * 0.5;
    }
}

// Предметы
for(let i = items.length-1; i>=0; i--) {
    if(items[i].update(dt)) items.splice(i, 1);
}

// Частицы
for (let i = particles.length - 1; i >= 0; i--) {
    particles[i].update(dt);
    if (particles[i].life <= 0) particles.splice(i, 1);
}

// Плавающий текст
for (let i = floatingTexts.length - 1; i >= 0; i--) {
    floatingTexts[i].y += floatingTexts[i].vy * (dt/16);
    floatingTexts[i].life -= dt;
    if (floatingTexts[i].life <= 0) floatingTexts.splice(i, 1);
}

// Камера плавно следит за игроком
camera.x += (player.x - width / 2 - camera.x) * 0.1;
camera.y += (player.y - height / 2 - camera.y) * 0.1;

// Затухание Screen Shake
if (screenShake > 0) screenShake *= 0.9;
if (screenShake < 0.5) screenShake = 0;
}

function draw() {
    // Очистка и заливка фона
    ctx.setTransform(1, 0, 0, 1, 0, 0);
    ctx.fillStyle = '#0a0a0a';
    ctx.fillRect(0, 0, width, height);

    ctx.save();

    // Применяем камеру и тряску
    let sx = (Math.random() - 0.5) * screenShake;
    let sy = (Math.random() - 0.5) * screenShake;
    ctx.translate(-camera.x + sx, -camera.y + sy);

```

```

// Рисуем сетку (землю) для понимания движения
ctx.strokeStyle = '#151515';
ctx.lineWidth = 2;
let gridSize = 100;
let startX = Math.floor(camera.x / gridSize) * gridSize;
let startY = Math.floor(camera.y / gridSize) * gridSize;
ctx.beginPath();
for (let x = startX - gridSize; x < camera.x + width + gridSize; x += gridSize) {
    ctx.moveTo(x, camera.y - 100); ctx.lineTo(x, camera.y + height + 100);
}
for (let y = startY - gridSize; y < camera.y + height + gridSize; y += gridSize) {
    ctx.moveTo(camera.x - 100, y); ctx.lineTo(camera.x + width + 100, y);
}
ctx.stroke();

// Декали крови
for (let d of bloodDecals) {
    ctx.fillStyle = `rgba(130, 0, 0, ${d.alpha})`;
    ctx.beginPath();
    ctx.arc(d.x, d.y, d.r, 0, Math.PI * 2);
    ctx.fill();
}

items.forEach(i => i.draw(ctx));
particles.forEach(p => p.draw(ctx));
enemies.forEach(e => e.draw(ctx));
bullets.forEach(b => b.draw(ctx));
player.draw(ctx);

// Плавающий текст (урон и тд)
ctx.font = "bold 16px Arial";
ctx.textAlign = "center";
for(let ft of floatingTexts) {
    ctx.fillStyle = ft.color;
    ctx.globalAlpha = Math.max(0, ft.life / ft.maxLife);
    ctx.fillText(ft.text, ft.x, ft.y);
}
ctx.globalAlpha = 1;

// Эффект виньетки / темноты вокруг (опционально, убрал для лучшей
производительности на мобилках)

ctx.restore();

// Отрисовка UI джойстиков ПОВЕРХ всего
ctx.setTransform(1, 0, 0, 1, 0, 0);
drawJoystick(input.leftJoy);

```



```

    drawJoystick(input.rightJoy);
}

function drawJoystick(joy) {
    if (!joy.active) return;

    // База джойстика
    ctx.beginPath();
    ctx.arc(joy.originX, joy.originY, 50, 0, Math.PI * 2);
    ctx.fillStyle = 'rgba(255, 255, 255, 0.1)';
    ctx.fill();
    ctx.strokeStyle = 'rgba(255, 255, 255, 0.3)';
    ctx.lineWidth = 2;
    ctx.stroke();

    // Стик
    let stickX = joy.originX + joy.dirX * joy.strength * 40;
    let stickY = joy.originY + joy.dirY * joy.strength * 40;
    ctx.beginPath();
    ctx.arc(stickX, stickY, 25, 0, Math.PI * 2);
    ctx.fillStyle = 'rgba(255, 255, 255, 0.5)';
    ctx.fill();
}

/**
 * ЗАПУСК И РЕСЕТ
 */
function startGame() {
    document.getElementById('startMenu').classList.add('hidden');
    initGame();
}

function resetGame() {
    document.getElementById('deathMenu').classList.add('hidden');
    initGame();
}

function initGame() {
    player = new Player();
    bullets = [];
    enemies = [];
    particles = [];
    bloodDecals = [];
    items = [];
    floatingTexts = [];

    score = 0;
    level = 1;

```

```
currentXP = 0;
xpToNextLevel = 10;
spawnRate = 1500;
lastSpawn = Date.now();

camera.x = player.x - width / 2;
camera.y = player.y - height / 2;

gameState = 'PLAYING';
lastTime = performance.now();
updateHUD();

requestAnimationFrame(gameLoop);
}

</script>
</body>
</html>

...

```